

AUTOM-02: Settings API

Series: AUTOM — Dynatrace Automation | **Notebook:** 2 of 8 | **Created:** January 2026 | **Last Updated:** 04/26/2026

The Settings API (also called Settings 2.0) is Dynatrace's modern REST API for configuration management. It provides a unified way to manage all Dynatrace settings through JSON objects with schema validation.

Table of Contents

- [1. Introduction](#)
 - [2. API Fundamentals](#)
 - [3. Working with Settings Objects](#)
 - [4. Common Operations](#)
 - [5. Best Practices](#)
-

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
API Token	Token with <code>settings.read</code> , <code>settings.write</code> scopes
Tenant URL	Your Dynatrace SaaS tenant URL
HTTP client	curl, Postman, or similar tool

Learning Objectives

By the end of this notebook, you will:

- Understand the Settings 2.0 API architecture
 - Know how to read and write configuration objects
 - Be able to list available schemas and their structure
 - Handle common API patterns and error cases
-

1. Introduction

Settings API vs Classic Config API

Aspect	Settings API (2.0)	Classic Config API
Format	Unified JSON schemas	Mixed endpoints
Validation	Schema-based	Varies by endpoint
New features	All new features	Legacy only
Recommended	Yes	Deprecated for new configs

Key Concepts

Concept	Description
Schema	Defines structure and validation rules for a config type
Object	An instance of configuration following a schema
Scope	Where the config applies (tenant, host, service, etc.)
Object ID	Unique identifier for a configuration object

Sprint 1.337 (April 2026) Updates

Configuration API → Settings v2 migration accelerated. As of Dynatrace SaaS sprint 1.337, many remaining Configuration API endpoints are now covered by Settings endpoints in Environment API v2. The legacy endpoints stay active for backward compatibility, but new automation should target the Settings v2 paths exclusively.

Extensions credentials gained `EXTENSION_AUTHENTICATION` `enum`. The Credentials API (both Environment and Config) now supports an `EXTENSION_AUTHENTICATION` scope value. This enables extension-based authentication scenarios — for example, granting a stored credential the ability to authenticate Extensions API calls without expanding its broader scope.

Extensions Action endpoint signature change. `PUT`

`/extensions/{extensionName}/monitoringConfigurations/{configurationId}/actions` now returns `agIds` (plural array) and deprecates the singular `agId` and `agName` properties. Update any code parsing the response to handle the array form.

Tokens: For Extensions automation, prefer **Platform tokens (`dt0s16` / `dt0s01`)** with `Authorization: Bearer ...` over the classic `dt0c01` (`Authorization: Api-Token ...`). The Platform token model is the recommended path going forward — see [AUTOM-08: Migration Automation](#) for the migration plan.

2. API Fundamentals

Base URL Pattern

```
https://{tenant-id}.live.dynatrace.com/api/v2/settings
```

Authentication

All requests require an API token in the `Authorization` header:

```
Authorization: Api-Token <your-api-token>
```

Core Endpoints

Endpoint	Method	Purpose
/api/v2/settings/schemas	GET	List available schemas
/api/v2/settings/schemas/{schemaId}	GET	Get schema definition
/api/v2/settings/objects	GET	List/search objects
/api/v2/settings/objects	POST	Create objects
/api/v2/settings/objects/{objectId}	GET	Get specific object
/api/v2/settings/objects/{objectId}	PUT	Update object
/api/v2/settings/objects/{objectId}	DELETE	Delete object

Listing Available Schemas

To see all available configuration types:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
-H "Authorization: Api-Token {token}" \
-H "Accept: application/json"
```

Common Schema IDs

Schema ID	Configuration Type
builtin:management-zones	Management zones
builtin:tags.auto-tagging	Auto-tagging rules
builtin:alerting.profile	Alerting profiles
builtin:problem.notifications	Problem notifications
builtin:anomaly-detection.hosts	Host anomaly detection
builtin:anomaly-detection.services	Service anomaly detection
builtin:slo	Service level objectives

3. Working with Settings Objects

Object Structure

Every settings object follows this structure:

```
{
  "schemaId": "builtin:management-zones",
  "scope": "environment",
  "value": {
    "name": "Production",
    "rules": [...]
  }
}
```

Field	Description
<code>schemaId</code>	The schema this object follows
<code>scope</code>	Where the config applies
<code>value</code>	The actual configuration data

Scope Types

Scope	Format	Example
Environment (tenant)	<code>environment</code>	<code>environment</code>
Host	<code>HOST-{id}</code>	<code>HOST-ABC123DEF456</code>
Host group	<code>HOST_GROUP-{id}</code>	<code>HOST_GROUP-XYZ789</code>
Service	<code>SERVICE-{id}</code>	<code>SERVICE-DEF456ABC</code>
Application	<code>APPLICATION-{id}</code>	<code>APPLICATION-123ABC</code>

Reading Objects

List all objects for a schema:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?
schemaIds=builtin:management-zones" \
-H "Authorization: Api-Token {token}"
```

Get a specific object:

```
curl -X GET
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
-H "Authorization: Api-Token {token}"
```

Query Parameters

Parameter	Description	Example
schemaIds	Filter by schema(s)	schemaIds=builtin:management-zones
scopes	Filter by scope(s)	scopes=HOST-ABC123
fields	Select specific fields	fields=objectId,value
pageSize	Results per page	pageSize=100

4. Common Operations

Create a Management Zone

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
-H "Authorization: Api-Token {token}" \
-H "Content-Type: application/json" \
-d '{
  "schemaId": "builtin:management-zones",
  "scope": "environment",
  "value": {
    "name": "Production-Web",
    "rules": [{
      "type": "SERVICE",
      "enabled": true,
      "conditions": [{
        "key": {
          "type": "STATIC",
          "attribute": "SERVICE_TAGS"
        },
        "comparisonInfo": {
          "type": "TAG",
          "operator": "TAG_KEY_EQUALS",
          "value": {
            "context": "CONTEXTLESS",
            "key": "environment"
          }
        },
        "negate": false
      }
    ]
  }
}]'
```

Create an Auto-Tagging Rule

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
-H "Authorization: Api-Token {token}" \
-H "Content-Type: application/json" \
-d '{
  "schemaId": "builtin:tags.auto-tagging",
  "scope": "environment",
  "value": {
    "name": "Application",
    "rules": [{
      "type": "SERVICE",
      "enabled": true,
      "valueFormat": "{Service:DetectedName}",
      "conditions": []
    }]
  }
}'
```

Update an Object

```
curl -X PUT
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
-H "Authorization: Api-Token {token}" \
-H "Content-Type: application/json" \
-d '{
  "value": {
    "name": "Production-Web-Updated",
    "rules": [...]
  }
}'
```

Delete an Object

```
curl -X DELETE
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
-H "Authorization: Api-Token {token}"
```


Bulk Operations

Create multiple objects in one request:

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
-H "Authorization: Api-Token {token}" \
-H "Content-Type: application/json" \
-d '[
  {"schemaId": "...", "scope": "...", "value": {...}},
  {"schemaId": "...", "scope": "...", "value": {...}},
  {"schemaId": "...", "scope": "...", "value": {...}}
]'
```

Error Handling

Common HTTP status codes:

Code	Meaning	Action
200	Success	Request completed
201	Created	Object created successfully
400	Bad Request	Check JSON syntax and schema
401	Unauthorized	Verify API token
403	Forbidden	Check token scopes
404	Not Found	Object or schema doesn't exist
409	Conflict	Object already exists
429	Rate Limited	Slow down requests

Validation Errors

The API returns detailed validation errors:

```
{
  "error": {
    "code": 400,
    "message": "Constraints violated",
    "constraintViolations": [
      {
        "path": "value.name",
        "message": "must not be blank"
      }
    ]
  }
}
```

5. Best Practices

API Token Security

Practice	Description
Least privilege	Only grant required scopes
Environment-specific	Separate tokens for dev/prod
Secret management	Use vaults, not hardcoded values
Rotation	Rotate tokens regularly

Rate Limiting

Limit Type	Recommendation
Requests per minute	Stay under 100 for bulk operations
Retry strategy	Exponential backoff on 429 errors
Batch operations	Use bulk endpoints where possible

Idempotency

Approach	Description
Check before create	Query for existing objects first
Use PUT for updates	PUT is idempotent, POST is not
Track object IDs	Store IDs for subsequent operations

Schema Discovery Pattern

Before creating objects, understand the schema:

1. **List schemas** to find the right one
2. **Get schema definition** for field requirements
3. **Review existing objects** for examples
4. **Validate** before bulk operations

```
# Step 1: Find the schema
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Api-Token {token}" | jq '.items[] | select(.schemaId | contains("management"))'

# Step 2: Get schema details
curl
"https://{tenant}.live.dynatrace.com/api/v2/settings/schemas/builtin:management-zones" \
  -H "Authorization: Api-Token {token}"

# Step 3: See existing examples
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?
schemaIds=builtin:management-zones" \
  -H "Authorization: Api-Token {token}"
```

6. Next Steps

When to Graduate from Settings API

Consider moving to higher-level tools when:

Scenario	Recommended Tool
Managing many configurations	Monaco
Need version control	Monaco or Terraform
Part of larger IaC pipeline	Terraform
Building a custom application	SDK

Continue the Series

Next Notebook	Focus
AUTOM-03: Monaco	Configuration-as-code with YAML

Additional Resources

- [Settings API Documentation](#)
- [API Token Management](#)
- [Schema Reference](#)

Summary

In this notebook, you learned:

- The Settings 2.0 API architecture and endpoints
- How to work with schemas, scopes, and objects
- Common CRUD operations for configuration
- Best practices for API token security and rate limiting

Key Takeaway: The Settings API is the foundation for all Dynatrace configuration automation. Understanding it helps you debug issues with higher-level tools like Monaco and Terraform.

Continue to AUTOM-03: Monaco to learn configuration-as-code patterns.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.